


# UNIVERSIDAD NACIONAL DE INGENIERÍA

## Facultad de Ciencias

## Curso: CC112 — Fundamentos de Programación

Docente: Mag. Americo Chulluncuy Reynoso

**2026-I**

# Recursividad e Iteración

Sesión 1 — CC112: Fundamentos de Programación

Facultad de Ciencias
Escuela Profesional de Ciencia de la Computación
Universidad Nacional de Ingeniería

# Tabla de Contenidos

1 Funciones en C++: Repaso esencial

2 Recursividad: Concepto y estructura

3 Problemas clásicos resueltos de forma recursiva

4 Recursividad vs. Iteración

5 Más ejemplos clásicos

6 Actividad práctica

# ¿Por qué necesitamos funciones?

## El problema del código monolítico

Un programa de 1000 líneas dentro de `main()` es:

- Difícil de **leer** y comprender
- Difícil de **mantener** y corregir
- Imposible de **reutilizar**
- Insostenible en proyectos reales

## Las funciones permiten

- ✓ Dividir el trabajo en tareas manejables
- ✓ Reutilizar código entre proyectos
- ✓ Facilitar las pruebas (*unit testing*)
- ✓ Fomentar el trabajo colaborativo

## Anatomía de una función en C++

```
// Prototipo (declaración)
double area(double b, double h);

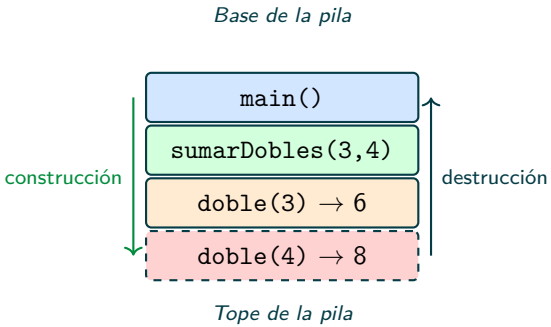
// Definición (implementación)
double area(double b, double h) {
    return b * h;
}

int main() {
    // Llamada (invocación)
    double r = area(5.0, 3.0);
    cout << r; // 15.0
}
```

# La pila de ejecución (*call stack*)

Cada llamada a una función **apila un marco** (*frame*) en memoria. Al retornar, ese marco se **destruye**.

```
int doble(int x) {
    return x * 2;
}
int sumarDobles(int a, int b) {
    return doble(a) + doble(b);
}
int main() {
    cout << sumarDobles(3, 4);
}
```




Resultado: `sumarDobles` retorna $6 + 8 = 14$ a `main()`.

# ¿Qué es la recursividad?

## Definición

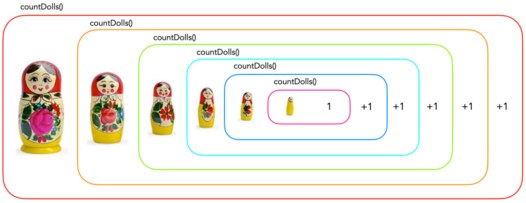
Una función es **recursiva** cuando se **llama a sí misma** para resolver una versión más pequeña del mismo problema.

Toda función recursiva bien formada tiene **dos partes obligatorias**:

**Caso base**
La condición que **detiene** la recursión.
Sin él ⇒ bucle infinito ⇒ *stack overflow*.

**Caso recursivo**
La llamada a sí misma con un **argumento reducido** que se acerca al caso base.

## Analogía: Muñecas rusas




## Estructura general

```
T funcion(T n) {
  if (caso_base)    // ← DETENER
      return valor_trivial;
  // ↓ REDUCIR y LLAMAR
  return
   ↪  combinar(n,funcion(reducir(n)));
}
```

# Ejemplo 1 — Factorial recursivo

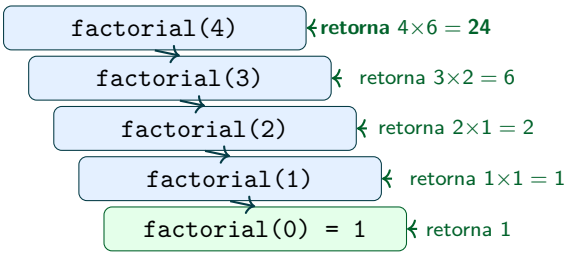
**Definición matemática:**

$$n! = \begin{cases} 1 & \text{si } n = 0 \\ n \times (n-1)! & \text{si } n > 0 \end{cases}$$

```c
int factorial(int n) {
    if (n == 0) // caso base
        return 1;
    return n * factorial(n - 1);
}
```

## Ejecución para `factorial(4)`

Hay **dos fases**: bajada (llamadas) y subida (retornos).



# Ejemplo 2 — Suma de los primeros $n$ naturales

**Definición matemática:**

$$\text{suma}(n) = \begin{cases} 0 & \text{si } n = 0 \\ n + \text{suma}(n-1) & \text{si } n > 0 \end{cases}$$

```
int suma(int n) {
    if (n == 0)       // caso base
        return 0;
    return n + suma(n - 1);
}

int main() {
    cout << suma(5); // 15
}
```

**Traza de ejecución para suma(5)**

| | |
|---|---|
| suma(5) <- | 5+10 = **15** |
| suma(4) <- | 4+6 = 10 |
| suma(3) <- | 3+3 = 6 |
| suma(2) <- | 2+1 = 3 |
| suma(1) <- | 1+0 = 1 |
| suma(0)=0 <- | retorna 0 |

# Ejemplo 3 — Fibonacci recursivo

**Definición matemática:**

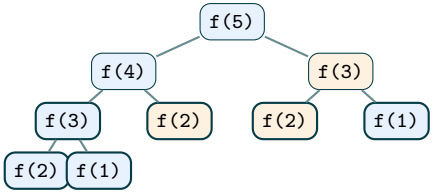
$$F(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ F(n-1) + F(n-2) & n > 1 \end{cases}$$

```c
int fib(int n) {
    if (n == 0) return 0; // base
    if (n == 1) return 1; // base
    return fib(n-1) + fib(n-2);
}
```

**Cuidado**

Fibonacci recursivo realiza **llamadas duplicadas**: $F(5)$ llama $F(3)$ **dos veces**, $F(2)$ **tres veces**, etc.

**Árbol de llamadas para `fib(5)`**




**Naranja**: subproblemas repetidos.

# El riesgo: *Stack Overflow*

### ⚠ Recursión sin caso base

Si la condición de parada está mal definida o ausente, la función se llama **infinitamente** hasta que la pila de ejecución se desborda.

```
// ERROR! no hay caso base
int malo(int n) {
    return n + malo(n - 1);
}
// Stack Overflow en tiempode ejecución
```

```
// ERROR! caso base inalcanzable
int malo2(int n) {
    if (n == 0) return 1;
    return malo2(n + 1); // ¡crece!
}
```

### Reglas de oro para la recursión

- **R1** El **caso base** debe existir siempre.
- **R2** Cada llamada debe acercarse **estrictamente** al caso base.
- **R3** El caso base debe ser **alcanzable** para todos los valores válidos.

### Versión correcta

```
int buena(int n) {
    if (n <= 0) return 0; // R1
    return n + buena(n-1);
    // n-1 < n  → R2 y R3
}
```

# Problema clásico 1 — Potencia $a^n$

**Definición:**

$$a^n = \begin{cases} 1 & n = 0 \\ a \times a^{n-1} & n > 0 \end{cases}$$

```
double potencia(double a, int n) {
    if (n == 0)       // caso base
        return 1.0;
    return a * potencia(a, n - 1);
}
```

## Uso

```
cout << potencia(2.0, 10); // 1024
cout << potencia(3.0,  4); //   81
```

## Versión optimizada

$$a^n = \begin{cases} 1 & n = 0 \\ (a^{n/2})^2 & n \text{ par} \\ a \cdot a^{n-1} & n \text{ impar} \end{cases}$$

```
double potRapida(double a, int n) {
    if (n == 0) return 1.0;
    if (n % 2 == 0) {
        double mitad =
            potRapida(a, n / 2);
        return mitad * mitad;
    }
    return a * potRapida(a, n - 1);
}
```

**Complejidad**: versión simple $O(n)$ vs $O(\log n)$.

# Problema clásico 2 — MCD (Algoritmo de Euclides)

**Propiedad de Euclides** (300 a.C.):

$$\text{mcd}(a, b) = \begin{cases} a & \text{si } b = 0 \\ \text{mcd}(b,\ a \bmod b) & \text{si } b > 0 \end{cases}$$

```
int mcd(int a, int b) {
    if (b == 0)    // caso base
        return a;
    return mcd(b, a % b);
}
```

## Uso

```
cout << mcd(48, 18); // 6
cout << mcd(100, 75); // 25
```

**Traza: `mcd(48, 18)`**

- `mcd(48, 18)` $48 = 2 \cdot 18 + 12$
- ↓
- `mcd(18, 12)` $18 = 1 \cdot 12 + 6$
- ↓
- `mcd(12, 6)` $12 = 2 \cdot 6 + 0$
- ↓
- `mcd(6, 0)` $\rightarrow$ **6**

Uno de los algoritmos más antiguos y elegantes de la historia de la computación. Complejidad: $O(\log \text{mín}(a, b))$.

# Problema clásico 4 — Búsqueda binaria recursiva

Buscar un elemento en un arreglo **ordenado** dividiendo el espacio a la mitad en cada llamada.

```
int busqBinaria(int arr[], int ini,
                int fin, int clave) {
    if (ini > fin) // caso base: no
        return -1;
    int mid = (ini + fin) / 2;
    if (arr[mid] == clave) // hallado
        return mid;
    if (clave < arr[mid])
        return busqBinaria(arr,
               ini, mid-1, clave);
    return busqBinaria(arr,
               mid+1, fin, clave);
}
```

## Traza: buscar 7 en {1,3,5,7,9,11,13}

Arreglo (índices 0–6)

| 1 | 3 | 5 | 7 | 9 | 11 | 13 |
|---|---|---|---|---|---|---|
| 0 | 1 | 2 | 3 | 4 | 5 | 6 |

mid = 3, arr[3]=7 → **¡encontrado!**

**Retorna índice 3. Solo 1 paso.**

Complejidad: $O(\log n)$ — mucho mejor que búsqueda lineal $O(n)$.

# Problema clásico 5 — Torres de Hanói

Mover $n$ discos de la torre **origen** a la torre **destino** usando una **auxiliar**, sin poner un disco grande sobre uno pequeño.

**Estrategia recursiva:**

1. Mover los $n-1$ discos de `A` a `C` (aux. `B`).
2. Mover el disco más grande de `A` a `B`.
3. Mover los $n-1$ discos de `C` a `B` (aux. `A`).

```
void hanoi(int n, char orig,
           char dest, char aux) {
 if (n == 1) {
    cout << orig << " " << dest << "\n";
    return;
 }
 hanoi(n-1, orig, aux,  dest);
 cout << orig << " " << dest << "\n";
 hanoi(n-1, aux,  dest, orig);
}
```

Salida para `hanoi(3,'A','B','C')`

```
A -> B
A -> C
B -> C
A -> B
C -> A
C -> B
A -> B
```

**Cantidad de movimientos**

$$M(n) = 2^n - 1$$

Para $n = 3$: $2^3 - 1 = 7$ movimientos.

Para $n = 64$: $\approx 1{,}8 \times 10^{19}$ movimientos.

# ¿Recursión o iteración? — Comparación general

| Característica | Recursión | Iteración |
|---|---|---|
| **Mecanismo** | Se llama a sí misma | Bucles `for`/`while` |
| **Memoria (stack)** | Mayor consumo | Menor consumo |
| **Legibilidad** | Más compacto | Más explícito |
| **Velocidad** | Generalmente más lenta | Más rápida |
| **Riesgo** | *Stack overflow* | Bucle infinito |
| **Depuración** | Más difícil | Más sencilla |

## Regla práctica

## Problema: Factorial de $n$

**Recursivo:**

```
int factR(int n) {
    if (n == 0) return 1;
    return n * factR(n - 1);
}
```

**Iterativo:**

```
int factI(int n) {
    int r = 1;
    for (int i = 2; i <= n;
    ↪  i++)
        r *= i;
    return r;
}
```

# Fibonacci: Recursivo vs. Iterativo

## Versión recursiva — $O(2^n)$

```
int fibR(int n) {
    if (n == 0) return 0;
    if (n == 1) return 1;
    return fibR(n-1) + fibR(n-2);
}
// fibR(40) tarda varios segundos
// fibR(50) puede tardar minutos
```

## Versión iterativa — $O(n)$

```
int fibI(int n) {
    if (n <= 1) return n;
    int prev=0, curr=1;
    for (int i=2; i<=n; i++) {
        int temp = curr;
        curr = prev + curr;
        prev = temp;
    }
    return curr;
}
// fibI(1000000) en microsegundos
```

**Llamadas realizadas por `fibR(n)` (aproximado)**



# Recursión de cola (*tail recursion*)

La **recursión de cola** ocurre cuando la llamada recursiva es la **última operación** de la función. Los compiladores modernos pueden optimizarla eliminando marcos del stack.

```
// NO es de cola: hay multiplicacion
// pendiente al retornar
int factR(int n) {
    if (n == 0) return 1;
    return n * factR(n-1); // ← n*...
}
```

```
// SÍ es de cola: el resultado
// se acumula en el parámetro
```

## ¿Por qué importa?

- En la versión normal: cada frame guarda el valor de `n` pendiente.
- En la versión de cola: el compilador **reutiliza** el mismo frame.
- Resultado: **memoria** $O(1)$ en lugar de $O(n)$.

## Llamada

```
cout << factTail(5);    // 120
cout << factTail(5, 1); // 120

// Traza:
```

# Conversión de recursivo a iterativo

## Principio general

Toda función recursiva puede convertirse a iterativa usando:

- Un **bucle** (`while` o `for`)
- Una **pila explícita** (cuando la recursión no es de cola)

**Suma de arreglo — recursivo:**

```
int sumaArr(int a[], int n) {
    if (n == 0) return 0;
    return a[n-1] + sumaArr(a, n-1);
}
```

**Suma de arreglo — iterativo:**

```
int sumaArrI(int a[], int n) {
    int total = 0;
    for (int i = 0; i < n; i++)
        total += a[i];
    return total;
}
```

## Fibonacci con memoización

Cuando la recursión tiene **subproblemas repetidos**, se puede guardar resultados en una tabla (*memoización*):

```
int memo[100] = {0};
int fibMemo(int n) {
```

# Suma de dígitos y contar dígitos

## Suma de dígitos de un número

$$\text{sumaDigitos}(n) = \begin{cases} n \\ (n \text{ mód } 10) + \text{sumaDigitos} \end{cases}$$

```
int sumaDigitos(int n) {
    if (n < 10) return n;
    return (n % 10) +
           sumaDigitos(n / 10);
}
// sumaDigitos(1234) = 10
// Traza: 4 + suma(123)
//           3 + suma(12)
//              2 + suma(1) = 1
```

## Número a binario

```
void aBinario(int n) {
    if (n == 0) return;
    aBinario(n / 2);
    cout << (n % 2);
}
// aBinario(13) → 1101
```

## Invertir una cadena

```
string invertir(string s) {
    if (s.length() <= 1)
        return s;
    return invertir(
```

# Resumen: ¿Cuándo usar cada enfoque?

**Usar recursión cuando...**

- El problema se define **naturalmente** de forma recursiva (factorial, Fibonacci, árboles).
- Se trabaja con **estructuras recursivas** (árboles binarios, listas enlazadas).
- Se aplica **divide y vencerás** (quicksort, mergesort, búsqueda binaria).
- La **claridad del código** es prioritaria sobre la eficiencia.

**Usar iteración cuando...**

- El proceso es **repetición simple** con contador.
- Se necesita **máxima eficiencia en memoria** y velocidad.
- La profundidad puede ser **muy grande** (riesgo de *stack overflow*).
- El compilador **no soporta** optimización de recursión de cola.

**Regla:** La recursión es una **forma de pensar** — elegante y poderosa. La iteración es una **forma de ejecutar** — eficiente y directa. El buen programador conoce ambas y elige según el contexto.

# Actividad práctica (Recursión vs iteración) — Sesión 1

1. **Potencia entera**: implementar $b^e$ de forma recursiva e iterativa.
2. **Suma de dígitos**: dado un entero $n$, retornar la suma de sus dígitos.
3. **Palíndromo**: verificar si una cadena leída por teclado es palíndromo.
4. **MCD**: calcular el máximo común divisor de dos enteros con el algoritmo de Euclides.
5. **Contar ocurrencias**: dada una cadena y un carácter, contar cuántas veces aparece el carácter.

## Desafío avanzado

6. **Torres de Hanói**: imprimir todos los movimientos para $n$ discos e indicar cuántos movimientos son en total.
7. **Búsqueda binaria**: implementar la versión recursiva sobre un arreglo de 10 enteros ordenados.
8. **Fibonacci con memoización**: implementar `fibMemo(n)` y comparar el tiempo con `fibR(n)` para $n = 35$.

## Consejo

Para cada función recursiva, trace a mano la ejecución para un valor pequeño antes de escribir el código.

# Próxima sesión

Algoritmos de ordenamiento y búsqueda I

achulluncuy@uni.edu.pe

Repositorio del curso

Aula virtual UNI

"To iterate is human, to recurse divine."

— L. Peter Deutsch

